Design Decisions & Revisions

Important Design Decisions

Prototype For Instance Creation

Problem: Each workflow instance needs its own deep copy of the definition's work tree. Two instances of the same definition must not share mutable state.

We rejected the Factory Method design pattern with one factory per item subclass because the engine would need to know every concrete task type, and adding a task would mean editing the factory.

WorkflowDefinition::createInstance() calls template_root->clone(). Every WorkItem subclass implements clone(). StageItem::clone() deep-copies every child; WorkItemDecorator::clone() clones the wrapped item and re-wraps it. CloneableTask<Derived> (CRTP helper) provides clone() automatically for any concrete task.

We have elected to do this because the definition's template tree is never bound or executed, it exists only to be cloned. createInstance() treats it as opaque. The clone contract is enforced at compile time (every concrete task inherits CloneableTask<T>), and isolation is testable (test_prototype.cpp mutates a clone and asserts the original is unchanged).

Uniform WorkItem Interface Instead of Role Interfaces

Problem: A WorkItem base class that owns state, structure, data, and memento would be a god-class. We needed to decide how the interface should be decomposed.

We rejected making use or role interfaces (such as StatefulWorkItem, CompositeWorkItem, ExecutableWorkItem) with separate contracts per role because every engine client (FlowController, DeadlineMonitor, Tui, ActionHistory) needs to treat tasks and stages uniformly, and role interfaces force downcasts at every call site.

We have one WorkItem interface with 26 methods, no logic lives on WorkItem, it is declarative. Instead, logic is distributed across collaborators:

- State transitions: WorkItemState (9 concrete states)
- Runtime data: WorkContext (bound via bind())
- Tree structure: StageItem (via children() and add())
- Memento: WorkItemMemento (originator is TaskItem)
- Description: WorkItemSpec (value object, shared by copy)

The interface is large but every method maps to one clear responsibility and the behaviour lives in the appropriate collaborator. A role-interface split would force callers to know which role they need, which evaluates to type checking. A uniform interface with

distributed logic lets the engine treat items identically while each concern keeps its own class.

State Pattern Applying Shared-Stateless-Object

Problem: Nine statuses and eleven lifecycle operations produce up to ninety legal/illegal cases. A switch (status) inside every TaskItem method would violate polymorphism rule and be unmaintainable.

We rejected a switch or if/else on status because not only is it bad programming practice, but a single switch inside ten methods would concentrate ninety cases in one place which is untestable in isolation and impossible to extend without editing the switch.

We have one WorkItemState subclass per status (CreatedState, AvailableState, AssignedState, InProgressState, UnderReviewState, EscalatedState, CompletedState, CancelledState, SkippedState) each exposing static instance() and holding no instance data. TaskItem holds a const WorkItemState* and delegates every lifecycle call. States are pure functions over (item, operation) to (new state, side effects), with static lifetime; TaskItem never deletes them.

We have elected to do this because shared-stateless-objects mean one allocation per state per process rather than one per task, and the pattern shape gives each state its own legal-transition table. The alternative of one state object per task would complicate cloning and require allocation per item; the alternative of an enum with free functions cannot represent per-state legality without a dispatch table.

Expire as a Lifecycle Transition, Not an Escalation

Problem: When a deadline lapses, we consider whether the item should escalate to a supervisor or complete with a default outcome (both occur in the two industries).

We rejected having a DeadlineMonitor publish DeadlineMissed then having it always be the case that FlowController responded by escalating. This was because it made the sequence of a merchant not responding in fourteen days resulting in default judgement for the customer impossible to express. A merchant's silence is a business outcome that routing should decide on, not an escalation to a supervisor.

We have WorkItem::expire(outcome_code, reason) as a new lifecycle operation. DeadlineMonitor still publishes DeadlineMissed; FlowController chooses based on WorkItemSpec::timeoutCode(). If a timeout code is set, the item expires with that code (a system completion with Outcome::rejected(code, reason) that routing can branch on). If the code is empty, the item escalates.

We have elected to do this because it makes timeouts routable without the item keeping a separate state to track whether escalation or expiry occurred. The banking workflow needs a business outcome from silence; the telecom workflow needs escalation from unreachability. Both fit the same mechanism once expire exists. The alternative, inline the

decision inside FlowController, would push business logic out of RoutingStrategy and break the separation that keeps routing configurable per industry.

Template Method & Memento in InstanceCommand

Problem: Every user action (start, submit, complete, approve, reject, escalate, cancel) needs the same sequence: find item, snapshot state, run action, restore on failure, mark executed on success. Written naively, seven commands would each implement that sequence.

We rejected a shared helper function because SubmitCommand and CompleteCommand need to validate the outcome before snapshotting, while CancelCommand and EscalateCommand do not. A free function cannot be overridden per command at the right point in the sequence.

We have InstanceCommand::execute() implementing the sequence once. Subclasses override only doExecute(item, actor), verb(), and detail(). The pre-action snapshot is an InstanceMemento held by the command; on failure the sequence restores immediately; on undo it restores and publishes an ItemReverted event.

We have elected to do this because Template Method puts the invariant sequence in one place, and Memento puts state capture in the originator where it belongs. Seven commands implementing the same sequence means seven places to fix when the sequence changes. Inheritance is the correct extension point here because the variation is what operation to perform, which is exactly what a primitive operation expresses.

MacroCommand With All-or-Nothing Execution

Problem: A user may want to perform several actions as one unit, for example complete three related tasks together. We considered whether steps one and two should be left completed if step three fails.

We rejected best-effort execution because it produces an unpredictable result. The user would have to track which step failed and decide whether to undo manually, defeating the purpose of wrapping the actions in a macro in the first place.

We have MacroCommand::execute() running its children in order. On the first failure it undoes already-executed children in reverse order (skipping children whose undo fails) and returns a failure identifying which step failed and how many were rolled back. All-success returns success.

We have elected to do this because a macro's purpose is to treat several actions as one, and partial success leaves the instance in a state the user did not intend. All-or-nothing matches the transactional semantics that a workflow user expects from a grouped command. The alternative of best-effort would make every macro's effect conditional on which step happened to succeed.

ActionHistory as Invoker Only, Audit via Events

Problem: Every state change must appear in the audit log. We considered where the log would be written.

We rejected our assumption from the planning phase, where ActionHistory was to be the audit-log writer, because many state transitions occur without a user command. FlowController::runAutomatic runs automatic items. DeadlineMonitor fires deadline events. FlowController publishes items during dependency release. If ActionHistory wrote the log, none of these transitions would be recorded, and the audit trail would be incomplete for exactly the events that matter most in a production system.

We have ActionHistory run commands and keep the undo stack, without writing the audit log. AuditObserver writes the log by observing WorkflowEvents published by the instance's EventPublisher. It is the single writer of the audit log.

We have elected to do this because writing via events covers every transition regardless of what caused it, and one writer observing one source is simpler than two writers coordinating. The alternative, both ActionHistory and FlowController writing directly, would create two writers and risk duplication and inconsistency. One source of truth for history is the correct design.

FlowController as Observer With a Re-entrancy Queue

Problem: FlowController reacts to events by publishing more events. A completed item's routing publishes successor items as available, which assigns them, which may auto-complete them, which routes further successors. Direct recursion overflows the stack on any real workflow.

We rejected direct recursion because it works only for toy workflows. A stage with N parallel children, each triggering a successor chain, produces a call stack proportional to the workflow depth, which exceeds the stack limit on workflows the demo uses.

We have FlowController::onEvent push events onto a pending deque. If the controller is not already draining, it drains in a loop, processing each event via react(). Reactions publish new events, which enqueue themselves at the back. Recursion is replaced by iteration.

We have elected to do this because the queue converts recursion into iteration bounded by the total number of events. A different design, a thread per instance, would introduce concurrency primitives the demo does not need, along with the risk of races. The queue is the correct serial solution: simple, deterministic, and testable.

ManualClock & Simulated Time in the Façade

Problem: SLA deadlines in both industries are measured in days or weeks. A live demonstration cannot wait for them, and tests cannot wait either.

We rejected real time with an optional override because it makes deadline demonstrations unpredictable, since the presenter would need to manipulate system time or wait for real time to pass. We also rejected no time manipulation at all because

the deadline paths would never be demonstrated, and the whole expire/escalation mechanism would be invisible in the demo.

We have Tividy require a ManualClock. Time moves only via advanceTime(seconds), which first advances the clock and then sweeps every running instance for lapsed deadlines. Each sweep publishes DeadlineMissed, which FlowController reacts to via expire or escalate.

We have elected to do this because it makes deadline behaviour demonstrable in five seconds: advance time by fourteen days, the merchant's response window lapses, the item expires, routing settles for the customer. That sequence takes a five-second demo instead of a two-week wait. It also makes deadline tests deterministic, since the test advances by exactly the timeout window and asserts the outcome. A production deployment would call advanceTime from a timer or replace the clock with a SystemClock, and the API is designed so either swap is trivial.

IndustryPack as Abstract Factory With Working Defaults

Problem: Each industry contributes a family of related products: a director, participants, connectors, an assignment strategy, an escalation chain, and a UI profile. If the pack declared all eight methods as pure virtual, every pack would be roughly two hundred lines of boilerplate.

We rejected requiring all methods to be pure virtual because a minimal pack, such as the DemoPack demonstrating document approval, has nothing to customise beyond its director, participants, and escalation chain. Requiring boilerplate implementations of the other five methods would obscure what each pack actually customises. We also rejected inlining the defaults in Tividy::installPack because that would bury the defaulting logic in the façade where it is hard to inspect.

We have three pure virtual methods on IndustryPack (key, displayName, createDirector) and five with working defaults: empty participants, one LoggingNotifier, RoleBasedAssignment, no escalation chain, and a neutral ClientProfile. A minimal pack is roughly fifty lines.

We have elected to do this because the defaults are correct for the simplest case: no participants, demo-visible notifications, role-based assignment, no automatic escalation. A pack overrides only what differs. This keeps each pack focused on its industry-specific behaviour and lets DemoPack serve as evidence that the engine runs with only generic classes when no industry-specific overrides are needed.

ExternalConnector as a Single-Method Adapter Target

Problem: Every external system, including core banking, card scheme, credit bureau, and network OSS, has its own API. We wanted to know how the Adapter interface should be shaped.

We rejected one method per operation because the interface would grow as vendors add operations, and every adapter would need to recompile when a new operation was

introduced for one vendor. We also rejected separate interfaces per vendor because the registry would be heterogeneous, with require(name) returning different types depending on what was requested, which defeats the purpose of a unified connector registry.

We have one method on ExternalConnector: invoke(ConnectorRequest) returning ConnectorResult. ConnectorRequest carries an operation name and a params map; ConnectorResult carries success, retryable, message, and a payload map. Each adapter dispatches on operation name internally. The ConnectorRegistry stores every connector behind the same interface.

We have elected to do this because the interface is tiny (one method, two value objects) and the registry is uniform. require(name) returns any connector, and every call site uses the same invoke shape. Adding an operation to one vendor does not touch the interface, does not require recompiling other adapters, and does not break the registry. The interface stability is what makes the Adapter pattern work in a system that will integrate with many external systems over its lifetime.

Two-Level Memento: Item and Instance

Problem: Undo needs to restore state. Here we consider the depth of the snapshot (one item or the whole instance).

We rejected item-only snapshots because a command may affect multiple items through a macro, or through routing consequences of completing one item, and the instance's WorkflowData (written to by adapters during performWork) plus its InstanceStatus also need restoring. An item-level snapshot alone misses these. We also rejected instance-only snapshots as the sole mechanism because single-item commands would waste memory copying every task's state on every action.

We have two levels. WorkItemMemento captures one task's state (status, assignee, counters, last outcome, who completed it), and only TaskItem, the originator, can create or read it. InstanceMemento captures all task mementos plus the instance's WorkflowData and InstanceStatus, and is created by WorkflowInstance::captureSnapshot(). InstanceCommand::execute() captures an InstanceMemento before the action and restores it on undo.

We have elected to do this because the two levels serve two different purposes. The instance-level snapshot is the unit of undo, because a command's effects may ripple across multiple items and the instance's shared data. The item-level memento is the building block that makes an instance-level snapshot possible without exposing every item's internal fields. Two levels, two concerns, one snapshot mechanism. The alternative of one level forced a choice between incompleteness and waste.

Tui::acting_as for Participant Impersonation

Problem: The engine is multi-user. The demonstration is single-user. The TUI must be able to attribute actions to different participants (customer, agent, analyst, senior analyst) without a real login system.

We rejected running one process per participant because the demo would require multiple terminals, and the presenter would have to switch windows to demonstrate a workflow that crosses roles. We also rejected a full login screen with password prompts because it adds nothing to the demo, since the point is to show the workflow, not to authenticate identities.

We have one Id acting_as field on Tui. Every action, every call to Tividy::perform(...), is attributed to acting_as. The "my work" screen prompts the user to enter a participant id, remembers it, and shows that participant's work list. Switching acting participants is a single input.

We have elected to do this because it is the minimum viable impersonation. The engine's permission checks and assignment logic operate on the actor id passed by the TUI, so switching acting_as exercises every branch of the security model. Acting as the wrong participant produces the same PermissionDeniedError a real user would see, and the TUI surfaces it as a controlled message. The alternative of a full authentication system would demonstrate nothing about the engine and would obscure the workflow under test.

Revisions

Date of Revision	Problem / Question Being Considered	Original Design / Assumption	Alternatives Considered	Change Made	Why Change Was Made	Updates to Relevant Diagrams
6 Oct '26	**Pattern Mapping**: pattern set grew from 12 to 15	Prototype, Visitor and Chain of Responsibility were not identified	N/A	Added them to the pattern table	Prototype for cloning a definition into instances, Visitor for reports, Chain of Responsibility for escalation	Pattern table was updated
6 Oct '26	**UML Diagrams**: known inconsistencies	Diagrams have been drawn conceptually from the scenario description before the engine exists, so there are inconsistencies across diagrams, but not across the concepts modelled by the diagrams	N/A	For future revisions based on an existing engine, we will fix the inconsistencies	The engine will settle the final names and inconsistencies, solving the constant changing of diagrams based on naming conventions of a conceptual, incomplete model open to future changes	Yes, once the engine is finalised diagrams will need to be updated to reflect this
12 Oct '26	**Git & GitHub**: GitHub Workflow	Was not initially established	N/A	A GitHub workflow has been established with automated checks, updated rulesets, etc. after first progress meeting with the TA	This makes collaboration more efficient, rules are strictly enforced and the repository is primarily self-managed	N/A
22 Oct '26	**Workflow Scenario**: Added a second workflow scenario and changed the first	Initially we had one workflow scenario	We considered selecting an industry for our original workflow scenario, but we	After progress meeting with the TA, we selected two workflow scenarios	The project requires two industry sections and the initial scenario was unable	We will need to update all diagrams

			could not select an appropriate one	across the Banking and Telecommunications industry because employee onboarding was not classifiable as one specific industry	to conform to an industry	which referenced the initial scenario and we will need to create diagrams for the second scenario, too
24 Oct ‘26	**Pattern Table**: Was previously written for onboarding and cited patterns cut during MVP	PA6 proposed 15 (later 17) patterns, including FailurePolicy strategy, second Chain of Responsibility, AuditedWorkItem, PriorityWorkItem, OverdueVisitor, EvidenceVisitor, DependentsIterator, AvailableWorkIterator, AnalyticsObserver, WorkloadAssignment, CompletionRule, AssignCommand	We considered confining ourselves to our initial design, however, it happens often that designs do not survive over to implementation.	Table rewritten against banking and telecom classes; eleven patterns cut and documented as DD-05 to DD-08; DD-02 Visitor reduced from three concrete visitors to one; Abstract Factory added via IndustryPack; final count 16 across 3 / 4 / 9 categories	The pattern table must match what it executes	This primarily affects the UML class diagram. This subsequently changed many report sections
24 Oct ‘26	**Scope & Requirements Revision**: PA6 scope and rationale claimed capabilities that the delivered design will not implement, the report was updated to be more	PA6 scope listed nine items including "repeated work" and "priority handling"; deferrals included "advanced analytics" and "automatic transfer between definition	N/A	"Repeated work" to "reworked up to a retry limit"; priority handling removed; deferrals updated to "analytics" and "multi-version definition storage" with version	TIVIDY remains functional without excessive requirements and the report needed to remove unsubstantiated PA6 claims that were falsifiable against the headers	N/A

	accurate when diffed with the deliverables	versions"		association retained per instance		
24 Oct ‘26	**Design Rationale**: Sections cited examples not in the delivered design	PA6 "Supporting Added Behaviour" listed priority handling; "Keeping Definition Versions Stable" claimed versioning was implemented; "Designing For Independent Variation" referred to a single scenario	N/A	"Supporting Added Behaviour" now names ValidatedWorkItem and SecureWorkItem and clarifies audit and notifications are event-level; "Keeping Definition Versions Stable" renamed to "Control of Changes to Process Definitions" and updated to reflect single-version library with per-instance version retention; "scenario" changed to "scenarios" throughout	Rationale must describe the delivered design, not the proposed one	N/A